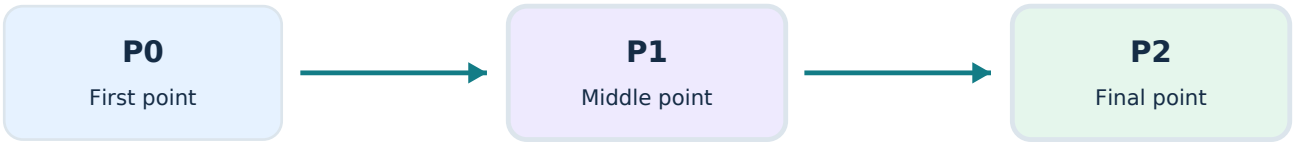
TELECOMMUNICATIONS PROTOCOL

A beginner-friendly, detailed explanation of modelling, invariants, Alloy verification, and reproducible evidence

Central question: When messages travel through multiple internal queues, how can we check that the visible submission and delivery histories still preserve the required ordering?



Messages move from left to right along the route

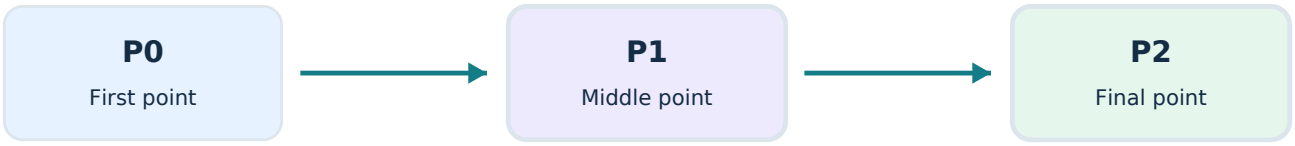
This guide walks through the same ideas as the earlier explanation: the external promise, internal queue representation, mathematical invariants, operations, retrieve relation, an actual two-message execution, Alloy checks, bounded results, and the evidence-validation pipeline.

Reading guide. Start with the example; then follow the equations. Every symbol is explained before it is used. “Correct within bounds” is intentionally kept separate from “universally proved.”

Source: Uploaded “Telecommunications protocol: the complete modelling and verification pipeline.” The figures in this guide are teaching illustrations; the recorded verification outcomes are taken from the supplied description.

1. The problem: messages must keep their order

Imagine A is submitted first, then B, then C. The network has several intermediate signalling points. Messages can wait at one point while others move forward. The required safety behaviour is that delivered messages must form the beginning of the chronological submission history - there must be no skipped or reordered deliveries.



Messages move from left to right along the route

This project does not merely test a few messages in a running application. It writes precise mathematical constraints and uses Alloy to search for counterexamples within finite search bounds.

2. Two connected workflows



Protocol workflow: A message is transmitted to the first point, forwarded through successive points by a daemon operation, and delivered from the last point. A route insertion operation can add an intermediate point.

Verification workflow: Define the expected behaviour, represent the internal implementation, check invariant preservation and external compatibility, save the exact model and evidence, validate results, then prepare reports.

3. External specification: what the observer can see

The external observer does not see intermediate queues. The observer sees only two ordered histories: input (all submitted message occurrences) and output (all delivered message occurrences).

> Unusual convention: The newest entry is written on the left. If A arrives, then B, then C, the input is `[C, B, A]`. If A and B are delivered in that order, the output is `[B, A]`.

```
input  = [C, B, A]
output = [B, A]   # valid: right-hand suffix
output = [C, A]   # invalid: B was skipped
```

The required safety rule is: output must be a suffix of input. A suffix is the rightmost portion of a sequence. The empty output is also a suffix: the safety rule permits messages to be delayed indefinitely, so it does not by itself promise eventual delivery.

4. Internal state: three histories for each point

The internal model introduces a route of signalling point codes (SPC) and a state snapshot. For each active point p, the model stores these sequences:

Field	What it records	Example at P0
rec[p]	Complete arrival history	[B, A]
ins[p]	Messages still pending at the point	[B]
sent[p]	Complete departure history	[A]

In this example, P0 received A and B. A has already left, but B is still waiting. The histories are linked to the identity of each point, not its route index, so inserting a new point does not rewrite another point’s past.

Occurrence versus value: The same message value can appear twice. A sequence such as [A,A] contains two occurrences even though only one distinct message atom is used. Their different sequence positions matter.

5. Internal invariants: the accounting rules

An invariant is a condition every valid state should satisfy. The model has structural requirements (for example, a nonempty route without duplicate points) and ordered accounting requirements.

Rule 1 - local queue accounting

```
rec[p] = ins[p] ++ sent[p]
[B, A] = [B] ++ [A]
```

The operator ++ means concatenate the sequences (put one list after the other); it is not arithmetic addition. This equation ensures every arrival belongs to either the pending queue or the departure history in the proper order. Matching counts alone would not be enough.

Rule 2 - agreement at neighbouring points

```
sent[upstream] = rec[downstream]
sent[P0] = [A]  and  rec[P1] = [A]
```

A departure recorded by P0 must appear as an arrival at the next point P1. These rules connect local operations to the order of the complete route.

> Why not assume every state is valid globally? The preservation checks assume the before state is valid, apply an operation, and ask whether the after state could violate the invariant. If validity were imposed on all states as a global fact, counterexamples would be excluded before Alloy could discover them.

6. The four principal operations

6.1 initial - start empty

The initial operation chooses a valid nonempty route and sets all received, pending, and sent sequences to empty. It is not restricted to exactly two points.

```
P0: rec=[], ins=[], sent=[]
P1: rec=[], ins=[], sent=[]
```

6.2 transmit - add a new submission

Transmitting A at the first point prepends A to both its arrival history and its pending queue. When B arrives next, those sequences become [B,A]. Other histories and the route are explicitly preserved.

```
After transmit A: rec[P0]=[A],   ins[P0]=[A]
After transmit B: rec[P0]=[B,A], ins[P0]=[B,A]
```

6.3 daemon - forward the oldest pending occurrence

The daemon sends one occurrence from a point to its immediate successor. Because new items are on the left, the rightmost element is the oldest and must move first.

Sequence	Before	After forwarding A
ins[P0]	[B, A]	[B]
sent[P0]	[]	[A]
rec[P1]	[]	[A]
ins[P1]	[]	[A]

Updating only the two pending queues would break the arrival/departure accounting. The daemon therefore updates all four relevant sequences and preserves fields not involved in the transfer.

6.4 receive and receivePoll - deliver or do nothing

Successful receive acts at the last point. For a pending queue [B,A], it removes A (the oldest) and prepends it to the final departure history. A second successful receive delivers B, making external output [B,A].

```
Before:  ins[last]=[B,A], output=[]
Receive A: ins[last]=[B],   output=[A]
Receive B: ins[last]=[],    output=[B,A]
```

The wrapper `receivePoll` permits either successful delivery or a stutter (an entirely unchanged state). Similarly, `daemonOrIdle` allows a forwarding step or no change. This matters: a system can repeatedly do nothing without violating the safety rules.

7. Inserting a route point without changing the visible history

Suppose the route changes from [P0,P1] to [P0,X,P1]. The new point X has no currently pending messages, but it must inherit the crossing history already shared by P0 and P1.

```
Old route:  P0 --> P1
New route:  P0 --> X --> P1
If [A] crossed the old boundary:
rec[X]  = [A]
ins[X]  = []
sent[X] = [A]
```

The new point does not physically travel back in time to handle A. These are historical accounting values so both new route boundaries remain consistent. For this reason, “empty inserted section” means its pending queue is empty, not that its arrival and departure histories must be empty.

8. Retrieve relation: how the detailed model matches the simple promise

A retrieve relation connects the detailed internal state to what an external observer sees. In a snapshot s:

```
input[s]  = rec[firstPoint[s]]
output[s] = sent[lastPoint[s]]
```

The model also constructs `inFlight` by concatenating the pending queues in route order. It then checks the stronger accounting relationship:

```
input = inFlight ++ output
Example: [C,B,A] = [C] ++ [B,A]
```

This directly explains the suffix property: delivered occurrences occupy the right-hand end of the complete input history, while the undelivered occurrences remain in flight. The abstraction checks also ensure that forwarding and insertion do not change the externally visible input, output, or total in-flight sequence.

This is bounded abstraction checking; the supplied project does not claim a complete unbounded data-refinement proof.

9. Full saved execution: two messages from start to finish

The following seven states correspond to the supplied saved Alloy witness with a two-point route. A and B are readable aliases for distinct message values. All sequences are newest-first.

Step	Action	P0 pending	P1 pending	Input	Output
0	Initialize	[]	[]	[]	[]
1	Transmit A	[A]	[]	[A]	[]
2	Transmit B	[B,A]	[]	[B,A]	[]
3	Forward A	[B]	[A]	[B,A]	[]
4	Forward B	[]	[B,A]	[B,A]	[]
5	Receive A	[]	[B]	[B,A]	[A]
6	Receive B	[]	[]	[B,A]	[B,A]

Forwarding changes where messages wait, but the visible input and output remain unchanged. Receiving A then B preserves order. At the end, no message is in flight and output equals input.

This execution shows a possible successful run; it does not prove every run will finish.

10. Alloy: SAT, UNSAT, examples, and counterexamples

Alloy searches for finite assignments of objects and relationships that satisfy the model. Two common commands have different purposes:

Command	Question asked	Meaning of SAT
run	Does a requested example exist?	An example was found within scope.
check	Can the assertion be violated?	A counterexample was found within scope.

For an intended safety assertion, a check returning UNSAT means Alloy found no counterexample within the stated bounds. For an intentional bug, SAT is expected, because a violation should be found.

The deliberately incorrect newest-first receive

The isolated buggy operation removes the leftmost (newest) pending message rather than the oldest. Consider this simple teaching example:

```
Before: rec=[B,A], ins=[B,A], sent=[]
Bug delivers B first:
After:  rec=[B,A], ins=[A], sent=[B]
Then:   ins ++ sent = [A,B] != [B,A]
```

The invariant fails. The actual saved Alloy bug witness includes repeated message values and is presented in the source report. The bug predicate is not part of the correct operation path.

11. Recorded outcomes and the finite search bounds

> Selected run: 18 outcomes matched expectations: 10 intended assertions returned UNSAT; 7 example runs returned SAT and saved witnesses; and the 1 intentional bug assertion returned SAT with its counterexample.

The provided run record identifies Alloy 6.2.0 and the glucose solver, with successful execution on 3 November 2026 and subsequent evidence validation on 4 November 2026.

Setting	Recorded bound
Distinct message values	At most 4 in correctness checks
Signalling point codes	At most 4
Maximum sequence length	4
State snapshots for correctness checks	1 or 2
Integers	Signed 5-bit
Complete delivery witness	Exactly 7 snapshots, 2 message values, 2 points

The sequence bound limits entire lifetime histories, not just currently pending messages. Delivering A does not remove A from the first point's arrival history; therefore, delivery does not free space for a fifth submission when lifetime histories are capped at four. The model uses guards to prevent sequence operations silently losing data at the bound.

> Interpretation: "No counterexample found" means none was found in the specified finite search space. It is not a universal guarantee for an unlimited number of points, messages, or operations.

12. Why the evidence-validation pipeline matters

Without reproducibility controls, a PASS report might accidentally refer to a different model revision, a partial run, or a stale result. The project therefore saves and verifies evidence before presenting it.

Freeze source: Create a unique results directory, copy the exact Alloy source into it, and compute its SHA-256 fingerprint.

Execute: Run Alloy on the frozen copy and record command, solver, versions, source and JAR hashes, timestamps, status, and exit code.

Save raw evidence: Keep Analyzer log output, command receipts, and XML instances for SAT results.

Validate before summarising: Check source hash, successful completion, command identities, order, expected results, bounds, solver agreement, overflow settings, and SAT evidence.

Decode examples carefully: Reject malformed sequence data and verify that saved examples really show the required deliveries or deliberate invariant failure.

The project also reports 14 passing regression tests for the reporting utilities, such as detecting changed sources, missing outcomes, malformed instances, failures, and interruptions. These tests support trustworthy reporting; they are distinct from Alloy's protocol assertions.

13. What the results support - and what they do not

Supported within recorded scope	Not established
Example transmissions, forwarding, insertion, and deliveries are possible.	Every message will eventually be delivered.
No counterexample was found for the ten intended finite checks.	Universal correctness for unbounded route/history sizes.
The deliberate incorrect receive can violate accounting.	Full real-world SS7 conformance or resilience to faults.
The reporting workflow validates saved execution evidence.	Real-time guarantees, fairness, retransmission, or complete deadlock freedom.

Safety says that incorrect deliveries should not occur. Liveness says that pending messages will eventually be delivered. Because stutter and idle steps are permitted, the safety checks by themselves cannot establish liveness.

14. The entire idea in four layers

Layer	Meaning
1. External contract	Input/output histories and the suffix-order promise.
2. Internal mechanism	SPC route, received/pending/sent histories, and protocol operations.
3. Formal checks	Invariants, retrieve relation, finite Alloy counterexample searches.
4. Reproducible evidence	Frozen source, execution logs, SAT witnesses, validation, and reports.

15. Three things to remember

1. Accounting: `input = inFlight ++ output` connects submitted, pending, and delivered occurrences without changing their ordering.

2. Verification: Start with a valid state, perform an operation, and search for a resulting state that breaks the required invariant.

3. Scope: UNSAT for the intended checks is strong bounded evidence, not an unbounded mathematical proof.

The core contribution described in the project is the precise and executable modelling of message ordering, together with a reproducible process for checking the model’s operations and reporting grounded results.